



Discrete Structures Final Project

Project Title: Smart Room Temperature Controller

Cohort A Group 9

Team Members: Kingsley Wunzooya Nahyi, Jachin Hugh Dzidumor Kpogli,
Emmanuel Nkunim Amoah Owusu-Marfo

Department of Computer Science

CS221: Discrete Structures and Theory

Mrs. Ayawoa Sitsopé Dagbovie

November 26, 2025

Introduction

The task to be completed is the building of an automated air-cooling system that factors in occupancy levels and temperature levels in rooms. Based on this, it assigns fan speeds to either low, high, or medium. Traditional cooling systems are human-controlled and involve manually turning a switch on or off. This is highly inefficient since it does not factor in occupancy levels, and it cools or warms at the same rate, independent of how many people are in the room at a particular time. This works on the principle of binary logic. Most real-world systems are not strictly “yes” or “no” but involve degrees of truth which cannot be conveyed by binary logic. For this reason, the design of this efficient cooling system using principles of fuzzy logic helps to curb this problem.

Fuzzy logic is an essential framework that allows variables to exist in different or multiple states to varying degrees, meaning it can be a member of different groups to a degree. Our design model uses inference rules to determine fan speeds. The system, operating on fuzzy logic, acknowledges that values are not always precise, and simple “if-then” rules do not suffice to deal with that. The project's relevance goes beyond just this application, as it can be used in other automation and artificial intelligence systems where fuzzy reasoning is necessary. This report details how this problem was solved independently using Python (a common programming language) and leveraging libraries like scikit-learn and matplotlib. The preceding paragraphs discuss the specific methods used in our program, our findings established from the plotted graphs, and the implications of these findings.

Method

To solve the problem of controlling fan speed based on temperature and occupancy using the fuzzy logic system, we took the following steps:

1. We imported the required libraries

```
1 import numpy as np
2 import skfuzzy as sk
3 from skfuzzy import control as ctrl
4 import matplotlib.pyplot as plt
5
```

- The library NumPy (numpy) was used to create arrays and generate mesh grids of temperature and occupancy values. i.e., generate every possible combination of temperature and occupancy values.
- The library scikit-fuzzy (skfuzzy) was used to define fuzzy variables, membership functions, rules, and to simulate the fuzzy control system.
- Control, which was imported from skfuzzy, allowed us to create fuzzy variables (Antecedents and Consequents), define fuzzy rules, and build and simulate the fuzzy control system.
- The library Matplotlib (matplotlib.pyplot) was used for visualizing the results in a 3D scatter plot.

2. We defined fuzzy variables

```
6 # antecedents or inputs coupled with creation of the universes or ranges for the various values
7 temperature = ctrl.Antecedent(np.arange(10, 41, 1), label='temperature')
8 occupancy = ctrl.Antecedent(np.arange(0, 21, 1), label='occupancy')
9
10 # consequent or outcome
11 fan_speed = ctrl.Consequent(np.arange(0, 101, 1), label='fan_speed')
```

We defined two inputs, “temperature” and “occupancy”, with the range 10 to 40°C and 0 to 20 people, respectively. We also defined one output, “fan_speed,” with a range of 0 to 100%. The inputs are what we refer to as Antecedents, and the outputs, Consequents, in fuzzy logic terminology.

3. We defined our membership functions

```
13 # definition of various membership functions for temperature
14 temperature['cool'] = sk.trapmf(temperature.universe, abcd=[10, 10, 18, 24])
15 temperature['comfortable'] = sk.trimf(temperature.universe, abc=[19, 24, 30])
16 temperature['warm'] = sk.trapmf(temperature.universe, abcd=[24, 30, 40, 40])
17
18 # definition of various membership functions of occupancy
19 occupancy['low'] = sk.trapmf(occupancy.universe, abcd=[0, 0, 5, 10])
20 occupancy['medium'] = sk.trimf(occupancy.universe, abc=[5, 10, 15])
21 occupancy['high'] = sk.trapmf(occupancy.universe, abcd=[10, 15, 20, 20])
22
23 # definition of various memberships for fan speed
24 fan_speed['low'] = sk.trapmf(fan_speed.universe, abcd=[0, 0, 30, 50])
25 fan_speed['medium'] = sk.trimf(fan_speed.universe, abc=[30, 50, 70])
26 fan_speed['high'] = sk.trapmf(fan_speed.universe, abcd=[50, 70, 100, 100])
```

Each fuzzy variable was assigned membership functions to define its fuzzy states. Trapezoidal functions (trapmf) were used for ranges with gradual transitions, and triangular functions (trimf) were used for ranges centered around a specific value. So, for example, “cool” temperature gradually ranges from 10 °C to 24 °C, whilst “comfortable” peaks at 24 °C, and “warm” starts from 24 °C to 40 °C.

4. We defined fuzzy rules

```
28 # rules for antecedents and outcomes.
29 rules = [
30     # Cool temperature rules
31     ctrl.Rule(temperature['cool'] & occupancy['low'], fan_speed['low']),
32     ctrl.Rule(temperature['cool'] & occupancy['medium'], fan_speed['low']),
33     ctrl.Rule(temperature['cool'] & occupancy['high'], fan_speed['medium']),
34
35     # Comfortable temperature rules
36     ctrl.Rule(temperature['comfortable'] & occupancy['low'], fan_speed['low']),
37     ctrl.Rule(temperature['comfortable'] & occupancy['medium'], fan_speed['medium']),
38     ctrl.Rule(temperature['comfortable'] & occupancy['high'], fan_speed['high']),
39
40     # Warm temperature rules
41     ctrl.Rule(temperature['warm'] & occupancy['low'], fan_speed['medium']),
42     ctrl.Rule(temperature['warm'] & occupancy['medium'], fan_speed['high']),
43     ctrl.Rule(temperature['warm'] & occupancy['high'], fan_speed['high'])
44 ]
```

The rules describe how the fan speed would behave with all the possible combinations of temperature and occupancy. For example, in our code, if the temperature is **cool** and occupancy is **low**, then the fan speed is **low**.

5. We built the fuzzy control system

```
46 # building the control system
47 fan_ctrl = ctrl.ControlSystem(rules)
48 fan_sim = ctrl.ControlSystemSimulation(fan_ctrl)
```

A controlSystem object was created from the rules. It stores all the rules and acts as a blueprint. ControlSystemSimulation allows simulation for specific input values of temperature and occupancy. i.e., we pass the ControlSystem object to the ControlSystemSimulation, which allows us to input specific values and compute the fuzzy output.

Steps 2 to 5 were performed in the function `getFuzzyCtrlSystem()`.

6. We asked for input and validated it.

```
75 def getInputValues():
76     print(" Smart Room Temperature Controller ".center(50, "*"))
77
78     while True:
79         try:
80             num_of_input_pairs = int(input("Enter the number of (temperature, occupancy level) pairs you would be entering: "))
81             if num_of_input_pairs <= 0:
82                 print("Please enter a positive integer value! Try again!")
83                 continue
84             break
85         except ValueError:
86             print("Please enter a valid integer value! Try again!")
```

```

88     temp_list = []
89     print("Enter your temperature values (10 - 40°C): ")
90     for i in range(num_of_input_pairs):
91         while True:
92             try:
93                 temp_value = float(input(f"{i+1}: "))
94                 if not (10 <= temp_value <= 40) :
95                     print("Temperature values must be in the allowed range from (10 - 40°C). Please Try again!")
96                     continue
97                 temp_list.append(temp_value)
98                 break
99             except ValueError:
100                 print("Please enter a valid integer/floating-point value! Try again!")
101
102     occ_list = []
103     print("Enter the occupancy levels (0 - 20): ")
104     for i in range(num_of_input_pairs):
105         while True:
106             try:
107                 occ_value = int(input(f"{i+1}: "))
108                 if not (0 <= occ_value <= 20) :
109                     print("Occupancy levels must be in the allowed range from (0 - 20 people). Please Try again!")
110                     continue
111                 occ_list.append(occ_value)
112                 break
113             except ValueError:
114                 print("Please enter a valid integer value! Try again!")
115

```

We requested user input in the function `getInputValues()`. This function asks how many pairs (temperature, occupancy) and then validates each value. For each entry, we check constraints. i.e., temperature must be 10-40°C and occupancy must be 0-20. This ensures that values entered by users fall inside the membership universes. We also store all valid values in NumPy arrays for convenience.

```

116         try:
117             temp_inputs = np.array(temp_list)
118             occ_inputs = np.array(occ_list)
119         except Exception as e:
120             print(e)
121
122         return temp_inputs, occ_inputs
123

```

7. We fuzzified and defuzzified

```

129         for i in range(len(temp_inputs)):
130             fan_sim.reset() # Safeguard to prevent leftover values
131             fan_sim.input['temperature'] = temp_inputs[i]
132             fan_sim.input['occupancy'] = occ_inputs[i]
133             fan_sim.compute()
134             fan_speeds[i] = fan_sim.output['fan_speed']

```

We evaluated each pair (temperature, occupancy) in the function `fuzzifyAndDefuzzify()`. “`fan_sim.reset()`” clears any previous inference state. This is very important when using the same simulator repeatedly. Lines 131 and 132 assign crisp inputs to the fuzzy system. “`fan_sim.compute()`” performs fuzzification. i.e., it

converts inputs to fuzzy sets. It also performs rule evaluation and defuzzification. We then store the defuzzified fan speed (0-100%). We then print a formatted table relating each pair to its computed output.

```
136     # Display a table of final outputs with its corresponding given inputs
137     print("\nPlotted Values")
138     print("Temperature (°C) --|--", "Occupancy Level (people) --|--", "Fan Speeds(%)")
139     for temp, occ, speed in zip(temp_inputs , occ_inputs, fan_speeds):
140         print(f"{temp:.1f} {occ:20.0f} {speed:29.0f}%")
141
142     return fan_speeds
143
```

8. We created 3D scatter plots.

```
145 def generateScatterPlot(temp_inputs, occ_inputs, fan_speeds):
146     # Create a figure
147     fig = plt.figure()
148     ax = fig.add_subplot(111, projection='3d')
149
150     # Create a scatter plot
151     ax.scatter(temp_inputs, occ_inputs, fan_speeds, c = fan_speeds, cmap='coolwarm', s=50)
152
153     # Customize labels and settings
154     ax.set_xlabel('Temperature (°C)')
155     ax.set_xlim(10, 40)
156     ax.set_ylabel('Occupancy level (number of people)')
157     ax.set_ylim(0, 20)
158     ax.set_yticks(range(0,21,4))
159     ax.set_zlabel('Fan speeds (%)')
160     ax.set_zlim(0, 100)
161     ax.set_title("Smart Room Temperature Controller")
162
163     # Show the plot
164     plt.show()
165
```

We created a 3D scatter plot with matplotlib. “ax” represents axes which reflect domain bounds (10- 40°C, 0- 20 people, 0-100%). The colour mapping (cmap) visually encodes fan intensity. “cmap = coolwarm” instructs Matplotlib to use the Coolwarm color map, indicating blue for lower fan speed and red for higher fan speed.

9. Main execution

```
168     # Main Program
169     if __name__ == "__main__":
170         fan_sim = getFuzzyCtrlSystem()
171         temperature, occupancy = getInputValues()
172         fan_speeds = fuzzifyAndDefuzify(temperature, occupancy, fan_sim)
173         generateScatterPlot(temperature, occupancy, fan_speeds)
```

This executes the whole code and calls all the various functions used earlier.

Results

To simulate the fuzzy logic behind a real-world Smart Room Temperature Controller, we tested our code with different temperature and occupancy level values, accepting input values through the terminal and displaying a confirmation table of the input values paired with their corresponding fan speeds, as well as the expected 3D scatter plot of the values.

We first tested our system with the expected values, that is, temperature (temp) integers or floating-point numbers in the range 10 – 40, as well as occupancy levels from 0 – 20. To ensure each temp value had a corresponding occupancy level, the system first requests the number of (temp, occupancy) pairs from the user before accepting the temp and occupancy input values. This helps prevent missing value errors during fuzzification in cases where the inputs are unevenly matched. An example of the expected case for input collection can be seen as follows;

```
***** Smart Room Temperature Controller *****
Enter the number of (temperature, occupancy level) pairs you would be entering: 10
Enter your temperature values (10 - 40°C):
1: 12
2: 36
3: 15
4: 40
5: 10
6: 21
7: 39
8: 25
9: 28
10: 30
Enter the occupancy levels (0 - 20):
1: 20
2: 20
3: 6
4: 0
5: 19
6: 8
7: 7
8: 16
9: 12
10: 11
```

Input values are accepted.

```
Plotted Values
Temperature (°C) --|-- Occupancy Level (people) --|-- Fan Speeds(%)
12.0             20             50%
36.0             20             80%
15.0             6              21%
40.0             0              50%
10.0             19             50%
21.0             8              31%
39.0             7              65%
25.0             16             79%
28.0             12             71%
30.0             11             79%
```

Table of input values with the corresponding fan speed percentages obtained through fuzzy logic

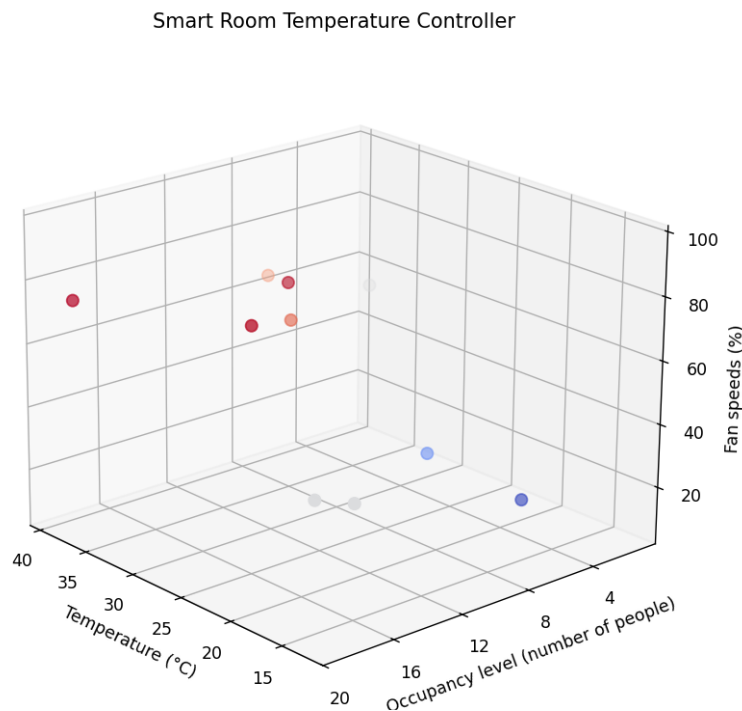


Fig. 1, Scatter plot of temperature, occupancy level, and fan speed values

These values were expected as they can be seen to be in line with the rules set in the fuzzy control system. Taking the first pair as an example, a temperature of 12°C is a cool temperature, and an occupancy level of 20 is considered to be a high occupancy level. Given the rules for the control system, a cool temperature coupled with a high occupancy level should give a medium fan speed. In this case, since the input values had membership values of 1 for their respective membership functions, it was expected that the fan speed was to be around 50% as seen.

Next, in our consideration of edge cases and error handling, the following were handled. Firstly, in the case the number of (temperature, occupancy level) pairs, the expected input is a positive integer and so if given zero, a negative number, a floating-point number, a letter or symbol or even if nothing is entered, an error is flagged to the user and an opportunity is given to the user to retry until a valid input value is provided. Also, zero is not included to prevent possible errors in the case that could occur if the control system were made to operate on empty arrays. An examination of how the system does this can be seen as follows;

```
while True:
    try:
        num_of_input_pairs = int(input("Enter the number of (temperature, occupancy level) pairs you would be entering: "))
        if num_of_input_pairs <= 0:
            print("Please enter a positive integer value! Try again!")
            continue
        break
    except ValueError:
        print("Please enter a valid integer value! Try again!")
```

Code handling input validation for the number of input pairs

```
Enter the number of (temperature, occupancy level) pairs you would be entering: 0
Please enter a positive integer value! Try again!
Enter the number of (temperature, occupancy level) pairs you would be entering: -3
Please enter a positive integer value! Try again!
Enter the number of (temperature, occupancy level) pairs you would be entering: a
Please enter a valid integer value! Try again!
Enter the number of (temperature, occupancy level) pairs you would be entering: @
Please enter a valid integer value! Try again!
Enter the number of (temperature, occupancy level) pairs you would be entering: 4
Enter your temperature values (10 - 40°C):
1: █
```

Demonstration of input validation for the number of input pairs (Terminal)

For the temperature values, similar to the pairs, invalid characters (letters, symbols, whitespaces) as well as nothing are flagged for the user to try again. Also, instead of all positive integers, an error is flagged if a given temp value falls outside the range of 10-40°C (inclusive). Temp values allowed could be integers or floating point numbers in this range. This is checked for each value entered, and if invalid, the system asks for a retry of that particular value until it is valid. This repetitive check during input collection, as compared to it being performed after everything is collected, helps prevent the user from giving all input values again for just a singular error. Also, collection for this range is ensured to prevent possible errors in the control system where values outside a given input variable's universe throw an error, and there is a successful flow of the program guaranteed from the start (during data collection).

Finally, errors related to the occupancy level were performed similarly to that for temperature, but the range was rather enforced for 0 to 20 people inclusive, where acceptable values were integers and not floating-point numbers (people can only be counted and cannot be floating-point). Below are some examples of how possible edge cases were handled and errors prevented.

```
Enter your temperature values (10 - 40°C):
1: a
Please enter a valid integer/floating-point value! Try again!
1: -1
Temperature values must be in the allowed range from (10 - 40°C). Please Try again!
1: 9.9
Temperature values must be in the allowed range from (10 - 40°C). Please Try again!
1: 41
Temperature values must be in the allowed range from (10 - 40°C). Please Try again!
1: 40.000
2: 11
3: 10.00001
4: █
```

Demonstration of Error handling for temperature values

```

Enter the occupancy levels (0 - 20):
1: 40
Occupancy levels must be in the allowed range from (0 - 20 people). Please Try again!
1: 13
2: a
Please enter a valid integer value! Try again!
2: %
Please enter a valid integer value! Try again!
2: -1
Occupancy levels must be in the allowed range from (0 - 20 people). Please Try again!
2: 21
Occupancy levels must be in the allowed range from (0 - 20 people). Please Try again!
2: 5
3: 

```

Demonstration of Error handling for occupancy level values

```

temp_list = []
print("Enter your temperature values (10 - 40°C): ")
for i in range(num_of_input_pairs):
    while True:
        try:
            temp_value = float(input(f"{i+1}: "))
            if not (10 <= temp_value <= 40):
                print("Temperature values must be in the allowed range from (10 - 40°C). Please Try again!")
                continue
            temp_list.append(temp_value)
            break
        except ValueError:
            print("Please enter a valid integer/floating-point value! Try again!")

occ_list = []
print("Enter the occupancy levels (0 - 20): ")
for i in range(num_of_input_pairs):
    while True:
        try:
            occ_value = int(input(f"{i+1}: "))
            if not (0 <= occ_value <= 20):
                print("Occupancy levels must be in the allowed range from (0 - 20 people). Please Try again!")
                continue
            occ_list.append(occ_value)
            break
        except ValueError:
            print("Please enter a valid integer value! Try again!")

```

Code handling input validation for temperature and occupancy level values

Discussion

In summary, our Smart Room Temperature Controller system leverages on fuzzy logic to determine the suitable fan speed for a room, given the measured temperature value as well as the number of people in the room. Such decision-making would have been very difficult, requiring complex mathematical models, but fuzzy logic simplifies this by building upon boolean logic to rely rather on a range from 0 to 1 to for truthfulness of a value, perfectly modeling how people approach problems.

Before we discuss our findings, understanding how our fuzzy logic system works is crucial. Taking into consideration the conversion for a single pair of input values, these crisp values (temperature and occupancy level) are passed as input. The crisp temperature value is

then converted into a set of membership functions that show the degree of to which it is cool, comfortable or warm. This is also done for the occupancy level ifor the degree to which it is low, medium or high. These sets are passed through an inference engine that also determines a fuzzy variable for the membership set specifying the degree to which the fan speed should be high, medium or low. This set is then defuzzified throug the method of centroid to obtain the easier to interpret fan speed value.

Taking Fig. 1 as an example, these values were expected as they can be seen to be in line with the rules set in the fuzzy control system. Taking the first pair as an example, a temperature of 12°C is a cool temperature, and an occupancy level of 20 is considered to be a high occupancy level. Given the rules for the control system, a cool temperature coupled with a high occupancy level should give a medium fan speed. In this case, since the input values had membership values of 1 for their respective membership functions, it was expected that the fan speed was to be around 50% as seen.

If given more time, some improvements we would make to our system are as follows. First of all, we would expand upon the range of input values accepted. The current system works well in a normal home where temperature usually does not go beyond this range but may fail if placed in a control system in places such as pharmaceutical cleanrooms, quantum computer and food processing chambers which require extremely cool environment temperatures, far below zero.

Secondly, to ensure crisp values produced as output are more accurate, we would include more membership functions and expand upon the rules of inference (inference engine) to fit the addition. This would make the system more effective in rooms used for semiconductor and microchip manufacturing where production is higly sensitive to environmental temperature. Last of all, we would incorporate a more modern user interface instead of the grim CLI in use. We would make use of a very simple progressive web app that takes values from the user. It will accept values being typed in the browser, as well as through data files such as csv and json. It will then process the output in the backend, display a table of the input and output values, as well as the scatter plot. An option would also be given for the table to be exported as an Excel file, also containing the scatter plot.

This project gave us several key insights into how fuzzy logic works and how it is potentially used in real-world control systems and machine learning. Also, we learnt that real world statements usually cannot be assigned a discrete boolean value of true or false but exists in a range of truthfulness, and hence in building systems, we must ensure appropriate logic systems are placed in place to be effective.

References

Fuzzy Control Systems: Advanced Example — *skfuzzy v0.2 docs*. (n.d.).

https://pythonhosted.org/scikit-fuzzy/auto_examples/plot_control_system_advanced.html#example-plot-control-system-advanced-py

GeeksforGeeks. (August 22, 2025). *Fuzzy Logic | Introduction*. GeeksforGeeks.

<https://www.geeksforgeeks.org/artificial-intelligence/fuzzy-logic-introduction/>

Jahan, M., & Jahan, M. (n.d.). *Basics of fuzzy logic*. Educative.

<https://www.educative.io/blog/basics-of-fuzzy-logic>

Scott, G. (July 1, 2025). *Fuzzy Logic: definition, meaning, examples, and history*.

Investopedia. <https://www.investopedia.com/terms/f/fuzzy-logic.asp>